

Speeding Up Paging

1 Core Problem

Main Issue

Paging is powerful, but it creates two implementation problems:

- address translation must be very fast;
- page tables can become very large.

Every instruction must be fetched from memory, and many instructions also read or write data in memory. Therefore virtual-address translation happens extremely often.

Why Speed Matters

If every memory access first requires a slow page-table lookup, paging can become the main performance bottleneck.

2 Why Page Tables Are Large

Modern machines use large virtual addresses. With a fixed page size, more address bits mean more virtual pages.

Address space	Effect on page table
32-bit virtual address	With 4-KB pages, there are about 2^{20} pages, or about one million page-table entries.
64-bit virtual address	The possible virtual address space is much larger, so a simple one-entry-per-page table becomes enormous.
Per process	Each process has its own address space, so each process needs its own page-table information.

Number of Virtual Pages

If the virtual address space has size S bytes and the page size is P bytes, then

$$\text{number of virtual pages} = \frac{S}{P}$$

3 Two Simple Designs

3.1 Page Table in Hardware Registers

Design

Store the whole page table in fast hardware registers, one register entry per virtual page.

Point	Explanation
Advantage	Translation is fast because the MMU does not need extra memory references to read page-table entries.
Problem 1	Large page tables require too many hardware registers, making the design expensive and impractical.
Problem 2	On every context switch, the operating system would have to load the full page table into registers.

3.2 Page Table in Main Memory

Design

Keep the page table in RAM. The CPU/MMU has a register pointing to the start of the current process's page table.

Point	Explanation
Advantage	Context switching is easier: the OS changes one page-table pointer register.
Problem	Each memory reference may require another memory reference to read the page-table entry.
Result	Simple and flexible, but too slow without a cache for translations.

The Basic Tradeoff

Hardware page tables are fast but too expensive for large address spaces. Memory-resident page tables are flexible but need help to avoid slow translation.

4 Translation Lookaside Buffer

Definition

A **Translation Lookaside Buffer (TLB)** is a small, fast hardware cache inside or near the MMU that stores recently used virtual-page to page-frame translations.

The TLB works because programs usually show **locality**: they repeatedly use a small set of pages for current code, data, and stack.

Key Intuition

Most memory references are not spread evenly over the whole page table. A small number of page-table entries are used again and again.

5 Typical TLB Entry

A TLB entry stores the information needed to translate and protect one virtual page.

Field	Meaning
Valid bit	Says whether this TLB slot currently contains a usable entry.
Virtual page number	The virtual page being cached. This is needed so the MMU can compare the requested page with the TLB entry.
Modified bit	Records whether the page has been written to. It may later be copied back to the page table.
Protection bits	Record allowed operations, such as read, write, and execute.
Page frame number	The physical frame where the page currently resides.

5.1 Example TLB Contents

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	RX	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	RX	50
1	21	0	RX	45
1	860	1	RW	14
1	861	1	RW	75

This kind of TLB might occur when a process is repeatedly executing code on pages 19, 20, and 21, using data on pages 129, 130, and 140, and using stack pages 860 and 861.

6 TLB Lookup

TLB First Rule

virtual page $\rightarrow$ check TLB first $\rightarrow$ $\begin{cases} \text{hit: use cached frame} \\ \text{miss: consult page table} \end{cases}$

When the CPU produces a virtual address, the MMU extracts the virtual page number and searches the TLB.

Case	What happens
TLB hit	The virtual page is found in the TLB. If protection allows the access, the MMU immediately uses the cached page frame number.
Protection fault	The page is in the TLB, but the requested operation is not allowed, such as writing to a read-only page. The CPU traps to the OS.
TLB miss	The virtual page is not in the TLB. The MMU or OS must look in the page table.

7 What Happens on a TLB Miss

7.1 If the Page Is Present in RAM

The page table contains a valid mapping, but that mapping was not cached in the TLB.

1. Look up the virtual page in the page table.
2. Choose a TLB entry to replace.
3. Copy the page-table entry into the TLB.
4. Restart or continue the memory reference.

Result

No disk access is needed. The page is already in RAM; only the TLB lacked the translation.

7.2 If the Page Is Not Present in RAM

The page table entry says the page is absent. This is a page fault.

1. Trap to the operating system.
2. Find or free a physical page frame.
3. Bring the missing page from disk into RAM.
4. Update the page table.
5. Update or refill the TLB entry.
6. Restart the faulting instruction.

Important Difference

A TLB miss is not automatically a page fault. A TLB miss means the translation is not in the TLB. A page fault means the page is not currently mapped in RAM, or the access is invalid.

8 Hardware TLB Management

Hardware-Managed TLB

In a hardware-managed TLB system, the MMU handles ordinary TLB misses by walking the page table and loading the needed TLB entry.

The operating system is normally involved only when the page is absent from RAM or when a protection violation occurs.

Event	Handler
TLB hit	MMU hardware handles it directly.
TLB miss, page present	MMU hardware performs a page-table walk and fills the TLB.
Page absent	OS handles the page fault.
Invalid or disallowed access	OS handles the fault, often by terminating the process.

9 Software TLB Management

Software-Managed TLB

In a software-managed TLB system, a TLB miss traps to the operating system. The OS finds the mapping, chooses a TLB slot, inserts the new entry, and restarts the instruction.

This approach was used on several RISC architectures. It simplifies the MMU because page-table searching is moved into operating-system software.

Advantage	Cost
Simpler MMU	TLB misses trap into the OS, so the miss handler must be very fast.
More OS control	Frequent misses can hurt performance.
More chip area for caches	The OS must carefully avoid extra TLB misses while handling a TLB miss.

10 Soft Misses and Hard Misses

Miss type	Meaning	Cost
Soft miss	Page is in RAM, but the translation is not in the TLB.	Fast: update the TLB; no disk I/O.
Hard miss	Page is not in RAM.	Slow: the OS must read the page from disk.
Minor page fault	Page is in RAM somewhere, but not mapped in this process's page table yet.	Slower than a soft TLB miss, but no disk read is needed.
Major page fault	Page must be fetched from disk.	Very slow compared with CPU instructions.
Invalid access	The process used an address it is not allowed to access.	OS raises an error such as segmentation fault or access violation.

Common Confusion

A hard miss is much more expensive than a soft miss because it needs disk I/O. A soft miss only repairs missing translation information.

11 Page Table Walk

Definition

A **page table walk** is the lookup procedure used to find the page-table entry for a virtual page.

With hardware TLB management, the MMU performs the walk. With software TLB management, the operating system may perform the walk after a TLB fault.

Why Walks Matter

If page-table pages themselves are not already easy to access, walking the page table can trigger additional translation work. This is one reason systems use caches and careful TLB miss handlers.

12 Reducing TLB Miss Cost

Systems may use several practical tricks:

Technique	Idea
Use a moderately large TLB	More entries reduce the chance that a useful translation has been evicted.
Preload likely entries	The OS may load entries for code, data, or stack pages that are likely to be used soon.
Software TLB cache	The OS can keep a software cache of mappings in a fixed location that is easy for the miss handler to access.
Exploit locality	Programs tend to reuse current code, data, and stack pages, so recent translations are valuable.

13 Big Picture

Mechanism	Main purpose	Main caveat
Page table	Complete mapping from virtual pages to page frames.	Large and too slow to consult on every access.
TLB	Small fast cache of recent translations.	Misses still need page-table lookup or OS handling.
Hardware TLB handling	Fast common-case miss handling by MMU.	More complex MMU hardware.
Software TLB handling	Simpler MMU and more OS control.	Miss handler must be extremely efficient.

Final Summary

Paging needs fast translation and compact page-table management.
The TLB speeds paging by caching recently used translations.
TLB miss $\neq$ page fault: the missing translation may still point to a page in RAM.